

pandas 데이터 재구조화(reshaping)

- 피벗팅(pivoting)
- 스택킹(stacking)과 언스택킹(unstacking)
- 멜팅(melting)과 와이드투롱(wide_to_long)
- 교차표(crosstab)
- explode

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
print(f'Numpy:{np.__version__}, Pandas:{pd.__version__}, Seaborn:{sns.__version__}')
```

Numpy:2.5.2, Pandas:3.0.5, Seaborn:0.13.2

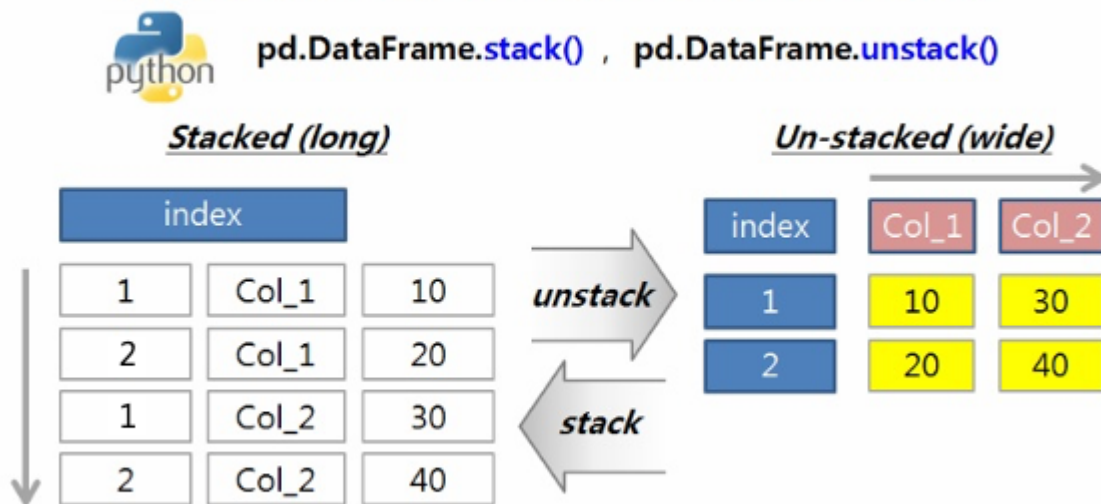
```
In [2]: from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
```

2. 스택킹(stacking)과 언스택킹(unstacking)

: 피벗팅과 유사하지만 계층형 인덱스의 특정 수준도 회전이 가능함

- 스택킹(stacking) : column labels과 그 값을 row index와 값으로 회전시킴
- 언스택킹(unstacking) : row index와 그 값이 column labels과 값으로 회전시킴

데이터 재구조화 (Reshaping data by stack, unstack)

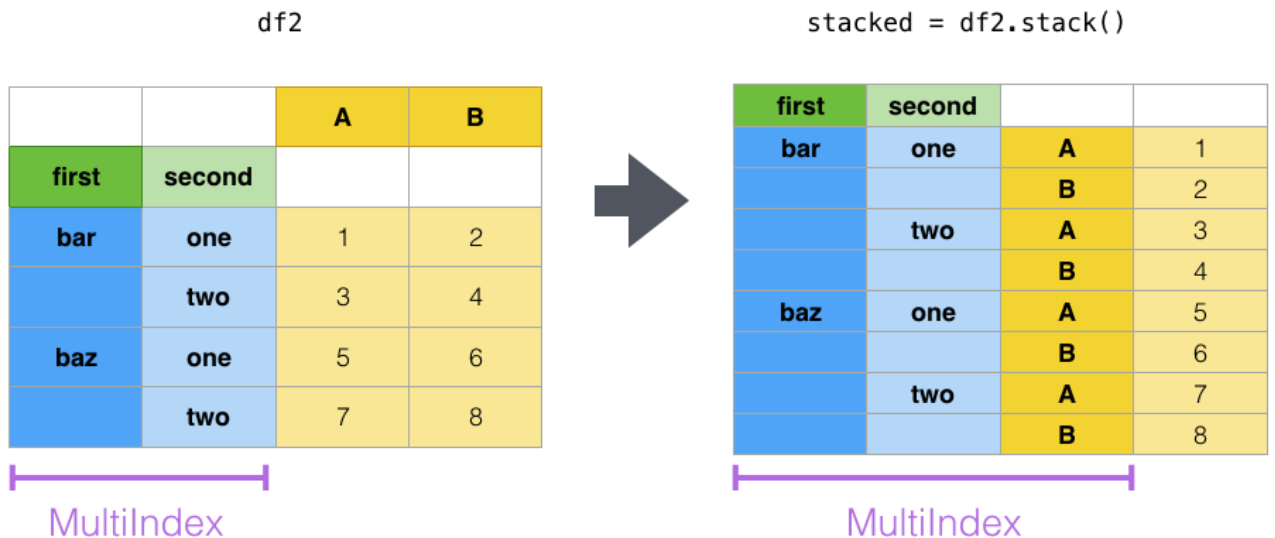


R, Python 데이터 분석과 프로그래밍의 친구 <http://rfriend.tistory.com>

- 출처 : <https://rfriend.tistory.com/276>

1. 스택킹(stacking)

Stack



- **DataFrame.stack(level=-1, dropna=<no_default>, sort=<no_default>, future_stack=True)**
 - level : int, str, list, default=-1
 - 언스태킹을 적용하는 레벨
 - 기본값은 마지막 레벨 : 언스태킹 결과 항상 마지막 레벨로 이동
 - dropna : bool, default True
 - 스택킹 결과 결측치 처리 여부, 기본값은 True로 결측치 제외
 - True : 결측값으로만 구성된 결과 행 제거
 - False : 결측값 행 유지
 - sort : MultiIndex 레벨을 정렬할지를 지정
 - True : 결과의 인덱스 레벨 정렬
 - False : 기존 열 순서를 최대한 유지
 - pandas 3.0부터 future_stack=True가 기본값. 따라서 일반적으로는 level만 지정하며, dropna와 sort는 지정하지 않는 것이 중요
 - future_stack=True일 때 dropna는 결과에 영향을 주지 않으며 지정하지 않아야 함
 - 특정 인덱스 레벨을 기준으로 정렬하려면 sort_index()사용

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.stack.html>

예제1. single level columns를 갖는 데이터

```
In [3]: df = pd.DataFrame([[10,11],[12,13]],
                        index=['cat','dog'],
                        columns=['weight','height'])
df
```

```
Out[3]:
```

	weight	height
cat	10	11
dog	12	13

- stack() : 컬럼이 인덱스 마지막 레벨로 변경 -> 시리즈 데이터로 변환

```
In [4]: df.stack()
```

```
Out[4]: cat  weight    10
         height    11
         dog  weight    12
         height    13
         dtype: int64
```

예제2. multi-level columns을 갖는 데이터1

```
In [5]: muticol = pd.MultiIndex.from_tuples([('weight', 'kg'),
                                             ('weight', 'pounds')])
df2 = pd.DataFrame([[10,11],[12,13]],
                    index=['cat','dog'],
                    columns=muticol)
df2
```

```
Out[5]:
```

	weight	
	kg	pounds
cat	10	11
dog	12	13

- stack() : 컬럼의 마지막 레벨이 인덱스의 마지막 레벨로 이동

```
In [6]: df2.stack()
```

```
Out[6]:
```

	weight	
	kg	pounds
cat	kg	10
	pounds	11
dog	kg	12
	pounds	13

```
In [7]: df2.stack(level=0)
```

```
Out[7]:
```

		kg	pounds
cat	weight	10	11
dog	weight	12	13

예제3. multi-level columns을 갖는 데이터2

```
In [8]: muticol2 = pd.MultiIndex.from_tuples([('weight', 'kg'),
                                              ('height', 'm')])
df3 = pd.DataFrame([[10,1.1],[12,1.3]],
                    index=['cat','dog'],
                    columns=muticol2)
df3
```

Out[8]:

	weight	height
	kg	m
cat	10	1.1
dog	12	1.3

- stack(level=-1)

```
In [10]: df3.stack()  
df3.stack(-1)
```

Out[10]:

		weight	height
cat	kg	10.0	NaN
	m	NaN	1.1
dog	kg	12.0	NaN
	m	NaN	1.3

Out[10]:

		weight	height
cat	kg	10.0	NaN
	m	NaN	1.1
dog	kg	12.0	NaN
	m	NaN	1.3

- stack(0) : 컬럼의 첫번째 레벨이 인덱스의 마지막 레벨로 이동

```
In [11]: df3.stack(0)  
df3.stack(level=0)
```

Out[11]:

		kg	m
cat	weight	10.0	NaN
	height	NaN	1.1
dog	weight	12.0	NaN
	height	NaN	1.3

Out[11]:

		kg	m
cat	weight	10.0	NaN
	height	NaN	1.1
dog	weight	12.0	NaN
	height	NaN	1.3

- stack([0,1]) : 컬럼의 두 레벨이 인덱스의 마지막 두 레벨로 이동

```
In [12]: df3.stack([0,1])
```

```
Out[12]: cat weight kg 10.0
          height m 1.1
          dog weight kg 12.0
          height m 1.3
          dtype: float64
```

예제4. multi-level columns을 갖는 데이터3 : 결측치를 포함하는 경우

```
In [13]: muticol2 = pd.MultiIndex.from_tuples([('weight','kg'),
                                                ('height','m')])
df4 = pd.DataFrame([[None,1.1],[12,1.3]],
                    index=['cat','dog'],
                    columns=muticol2)
df4
```

```
Out[13]:
```

	weight height	
	kg	m
cat	NaN	1.1
dog	12.0	1.3

- stack()

```
In [15]: df4.stack()
```

```
Out[15]:
```

	weight height		
	cat kg	NaN	NaN
	m	NaN	1.1
	dog kg	12.0	NaN
	m	NaN	1.3

- stack(, dropna=True|False)

```
In [18]: # df4.stack(dropna=False)
# 행에 모두 결측치인 경우 그 행을 삭제 :
# future_stack=False인 경우 적용하나, 사용하지 말것을 권장(3.0버전 이후부터)
df4.stack(future_stack=False, dropna=True)
```

C:\Users\acorn2\AppData\Local\Temp\ipykernel_18472\1486499146.py:4: Pandas4Warning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Do not specify the future_stack argument to adopt the new implementation and silence this warning.

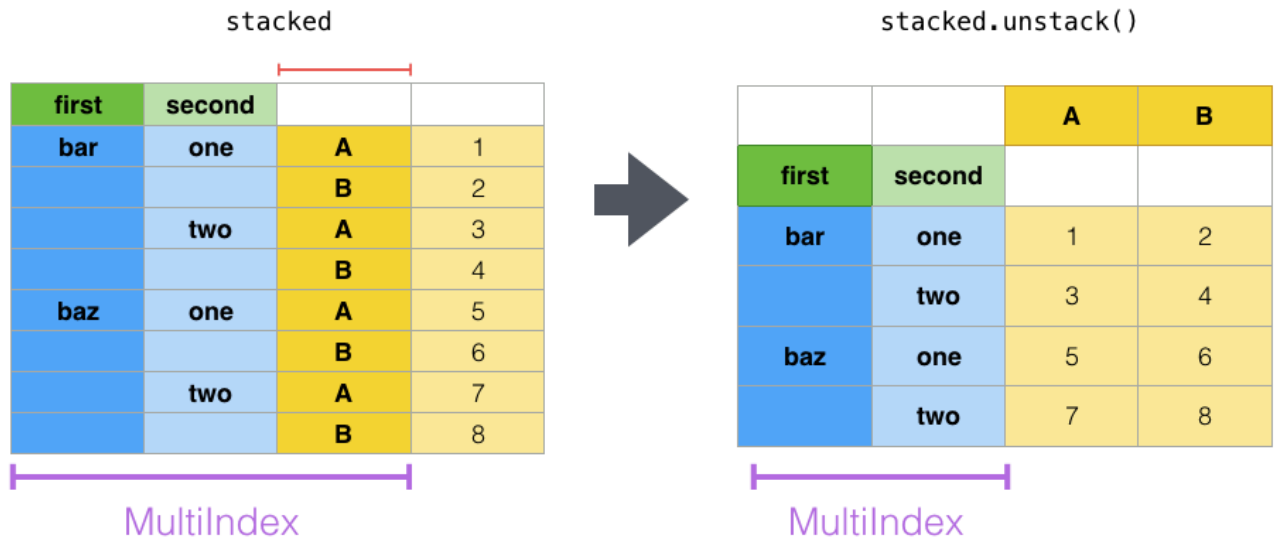
```
df4.stack(future_stack=False, dropna=True)
```

```
Out[18]:
```

	weight height		
	cat m	NaN	1.1
	dog kg	12.0	NaN
	m	NaN	1.3

2. 언스태킹(unstacking)

Unstack



- **DataFrame.unstack(level=-1, fill_value=None, sort=True)**

- level : int, str, list, default=-1
 - 언스태킹을 적용하는 레벨
 - 기본값은 마지막 레벨 : 언스태킹 결과 항상 마지막 레벨로 이동
- fill_value : int, str or dict
 - 언스태킹 결과 결측치는 NaN으로 대체
- sort : bool, default True
 - 멀티인덱스 컬럼의 레벨

- <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.unstack.html>

- 예제5. 시리즈 데이터

```
In [19]: idx = pd.MultiIndex.from_tuples([('one', 'a'), ('one', 'b'),  
                                         ('two', 'a'), ('two', 'b')])  
s = pd.Series(np.arange(1.0, 5.0), index=idx)  
s
```

```
Out[19]: one  a    1.0  
         b    2.0  
         two a    3.0  
          b    4.0  
dtype: float64
```

- unstack() : 마지막레벨로 unstacking

```
In [21]: s.unstack()  
s.unstack(-1)  
s.unstack(level=-1)
```

```
Out[21]:
```

	a	b
one	1.0	2.0
two	3.0	4.0

```
Out[21]:
```

	a	b
one	1.0	2.0
two	3.0	4.0

```
Out[21]:
```

	a	b
one	1.0	2.0
two	3.0	4.0

- `unstack(level=0)`

```
In [22]: s.unstack(level=0)
```

```
Out[22]:
```

	one	two
a	1.0	3.0
b	2.0	4.0

예제6. 행이 MultiIndex를 갖는 데이터

```
In [23]: lists = ['bar bar baz baz foo foo qux qux'.split(),
                  'one two'.split()*4]

lists
idx = pd.MultiIndex.from_arrays(lists, names=['first', 'second'])
df = pd.DataFrame(np.round(np.random.randn(8,2), 2),
                  index=idx, columns=['A', 'B'])

df
```

```
Out[23]:
```

['bar', 'bar', 'baz', 'baz', 'foo', 'foo', 'qux', 'qux'],
['one', 'two', 'one', 'two', 'one', 'two', 'one', 'two']]

```
Out[23]:
```

		A	B
bar	one	1.38	-1.11
	two	0.36	-0.56
baz	one	1.21	-0.32
	two	0.50	0.11
foo	one	-0.04	0.32
	two	-0.10	-0.95
qux	one	-0.24	-1.40
	two	-0.67	-0.77

- `unstack()` : index의 마지막레벨이 컬럼의 마지막 레벨로 이동

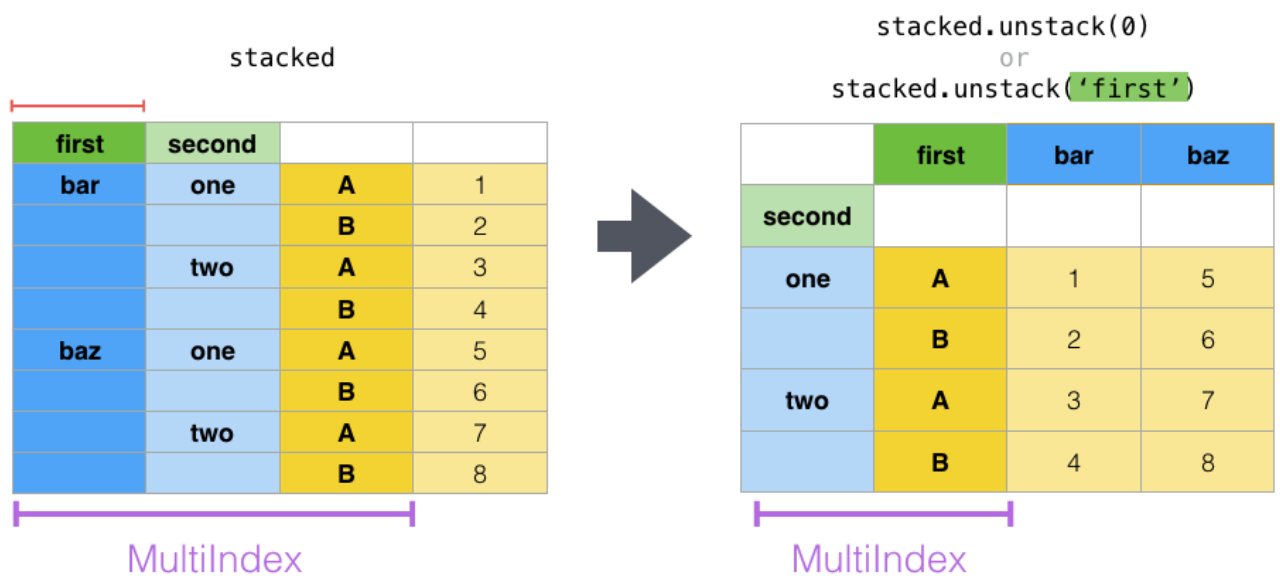
```
In [24]: df.unstack()
```

```
Out[24]:
```

	A				B			
second	one	two	one	two	one	two	one	two
first								
bar	1.38	0.36	-1.11	-0.56				
baz	1.21	0.50	-0.32	0.11				
foo	-0.04	-0.10	0.32	-0.95				
qux	-0.24	-0.67	-1.40	-0.77				

- `unstack(0)` : index의 첫번째 레벨이 컬럼의 마지막 레벨로 이동

Unstack(0)



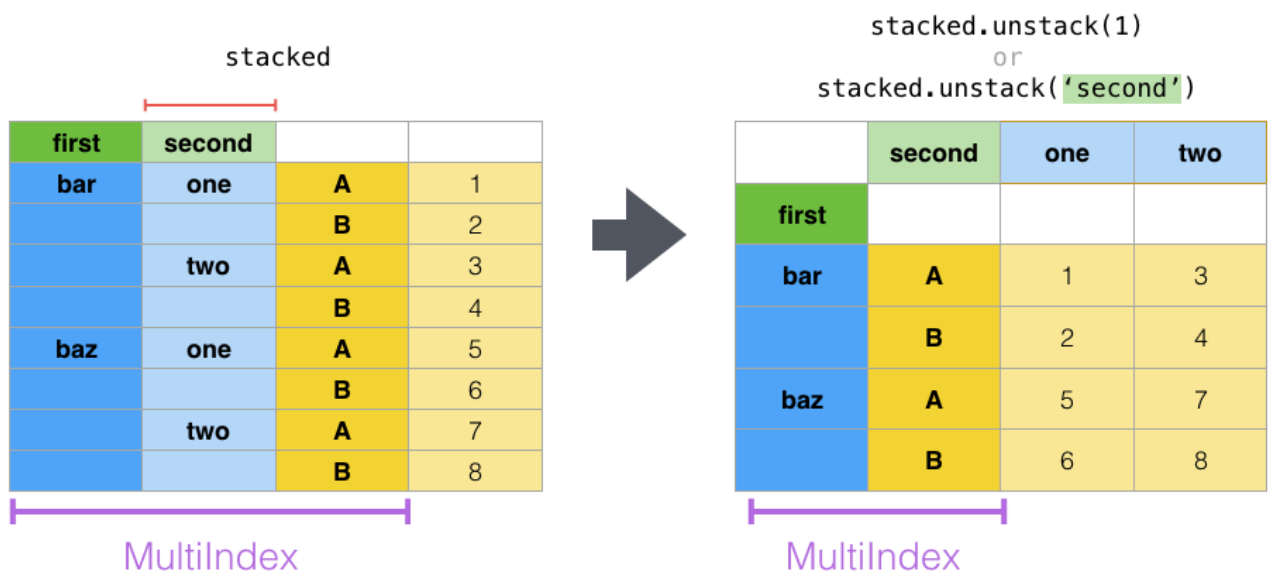
```
In [25]: df.unstack(0)
```

```
Out[25]:
```

	A				B			
first	bar	baz	foo	qux	bar	baz	foo	qux
second								
one	1.38	1.21	-0.04	-0.24	-1.11	-0.32	0.32	-1.40
two	0.36	0.50	-0.10	-0.67	-0.56	0.11	-0.95	-0.77

- `unstack(1)` : index의 두번째 레벨이 컬럼의 마지막 레벨로 이동

Unstack(1)



```
In [27]: df.unstack(1)
df.unstack()
df.unstack(-1)
```

```
Out[27]:
```

	A		B	
second	one	two	one	two
first				
bar	1.38	0.36	-1.11	-0.56
baz	1.21	0.50	-0.32	0.11
foo	-0.04	-0.10	0.32	-0.95
qux	-0.24	-0.67	-1.40	-0.77

```
Out[27]:
```

	A		B	
second	one	two	one	two
first				
bar	1.38	0.36	-1.11	-0.56
baz	1.21	0.50	-0.32	0.11
foo	-0.04	-0.10	0.32	-0.95
qux	-0.24	-0.67	-1.40	-0.77

Out[27]:

	A		B	
second	one	two	one	two
first				
bar	1.38	0.36	-1.11	-0.56
baz	1.21	0.50	-0.32	0.11
foo	-0.04	-0.10	0.32	-0.95
qux	-0.24	-0.67	-1.40	-0.77

```
In [28]: df.unstack('first')
df.unstack('second')
```

Out[28]:

				A			B	
first	bar	baz	foo	qux	bar	baz	foo	qux
second								
one	1.38	1.21	-0.04	-0.24	-1.11	-0.32	0.32	-1.40
two	0.36	0.50	-0.10	-0.67	-0.56	0.11	-0.95	-0.77

Out[28]:

	A		B	
second	one	two	one	two
first				
bar	1.38	0.36	-1.11	-0.56
baz	1.21	0.50	-0.32	0.11
foo	-0.04	-0.10	0.32	-0.95
qux	-0.24	-0.67	-1.40	-0.77

예제7. 행과 열이 모두 MultiIndex를 갖는 데이터

```
In [29]: idx = pd.MultiIndex.from_product(['bar baz foo qux'.split(), ('one', 'two')],
names=['first', 'second'])
cols = pd.MultiIndex.from_tuples([('A', 'cat'), ('B', 'dog'), ('B', 'cat'), ('A', 'dog')],
names=['category', 'animal'])
df = pd.DataFrame(np.round(np.random.randn(8, 4), 2),
index=idx, columns=cols)
df
```

Out[29]:

category		A		B		A
animal		cat	dog	cat	dog	
first	second					
bar	one	0.88	0.47	-0.58	0.66	
	two	0.57	-0.65	1.66	0.83	
baz	one	-1.93	2.81	-0.96	0.96	
	two	0.02	-0.08	-0.77	-0.89	
foo	one	2.85	0.06	-0.22	0.59	
	two	0.56	-0.08	2.09	0.96	
qux	one	-0.68	0.08	-0.70	0.59	
	two	-0.38	-0.32	1.13	0.29	

In [30]: `df.loc[('bar','one'),('A','cat')]`

Out[30]: `np.float64(0.88)`

In [31]: `df.unstack()`

Out[31]:

category		A				B		A	
animal		cat		dog		cat		dog	
second	one	two	one	two	one	two	one	two	
first									
bar	one	0.88	0.57	0.47	-0.65	-0.58	1.66	0.66	0.83
	two	-1.93	0.02	2.81	-0.08	-0.96	-0.77	0.96	-0.89
foo	one	2.85	0.56	0.06	-0.08	-0.22	2.09	0.59	0.96
	two	-0.68	-0.38	0.08	-0.32	-0.70	1.13	0.59	0.29

In [33]: `df.iloc[[0,1,4,7],[1,2]]`
`df.iloc[[0,1,4,7],[1,2]].unstack()`

Out[33]:

category		B	
animal		dog	cat
first	second		
bar	one	0.47	-0.58
	two	-0.65	1.66
foo	one	0.06	-0.22
qux	two	-0.32	1.13

Out[33]:

category				B
animal		dog		cat
second	one	two	one	two
first				
bar	0.47	-0.65	-0.58	1.66
foo	0.06	NaN	-0.22	NaN
qux	NaN	-0.32	NaN	1.13

- `unstack( , fill_value=)`

In [34]: `df.iloc[[0,1,4,7],[1,2]].unstack(fill_value=0)`

Out[34]:

category				B
animal		dog		cat
second	one	two	one	two
first				
bar	0.47	-0.65	-0.58	1.66
foo	0.06	0.00	-0.22	0.00
qux	0.00	-0.32	0.00	1.13
